

Section 2: Architecture Overview

Context

This section describes Ferrite's system-level architecture. It defines the process model, trust boundaries, data flow between components, and how every module relates to every other module. This section is the architectural source of truth — all other sections must be consistent with it.

Core Architectural Principle

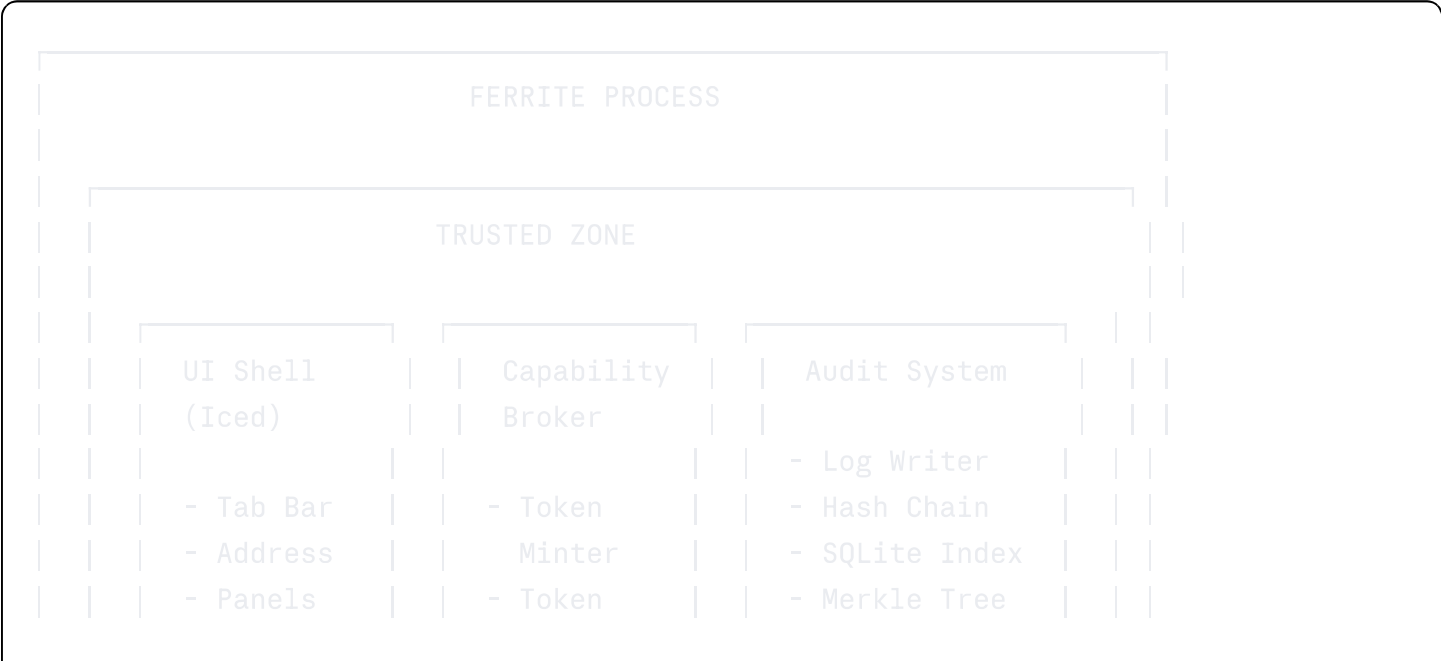
The Capability Broker is the single privileged boundary. All untrusted code (extensions, agents, web content) accesses all sensitive operations (DOM, network, storage, clipboard, downloads, notifications, accessibility tree) **ONLY** through the broker. The broker mints unforgeable capability tokens after policy evaluation and logs every decision to the audit system.

No ambient authority exists anywhere in the system. This is the fundamental difference from every other browser.

Process Model

Ferrite runs as multiple logical components within a process boundary. For the 8-month prototype, all components run in a single OS process using Tokio async tasks for concurrency. Process isolation (separate OS processes per component) is future work.

Components



- Consent
Dialogs

Validator
- Policy
Engine
(Regorus)
- Risk
Classifier

- Proof Gen

BROKER INTERFACE
(async channels)

UNTRUSTED ZONE

Servo
Engine

- Content
- JS (SM)
- Layout
- Render
(WR+wgpu)

Extension
Sandbox
(Extism/
Wasmtime)

- Wasm
Modules
- Host
Functions

Agent Runtime

- JSON-RPC/WS
Server
- Command
Router
- Consent
Callback
Handler

Network
Stack

- hyper
- rustls
- quinn
- adblock

CEF Harness
(Track B)

- cef-rs
- Same
broker
interface

Trust Boundaries

Boundary 1: Trusted Zone ↔ Untrusted Zone

This is the primary security boundary. The broker interface is the ONLY crossing point.

Trusted Zone contains:

- UI Shell (Iced) — renders consent dialogs, DevTools panels, tab bar, address bar
- Capability Broker — mints tokens, validates tokens, evaluates policies
- Audit System — writes and verifies the append-only log
- Policy Engine (Regorus) — evaluates Rego policies
- Risk Classifier — categorizes actions by risk level

Untrusted Zone contains:

- Servo engine — renders web content (potentially malicious)
- Extension sandbox — runs Wasm extensions (potentially malicious)
- Agent runtime — executes LLM-driven commands (potentially manipulated)
- Network stack — fetches content from the internet (potentially hostile)
- CEF harness — same trust level as Servo

Crossing rules:

- Untrusted → Trusted: only via capability requests (structured messages over async channels)
- Trusted → Untrusted: only via capability tokens (opaque, unforgeable, scoped)
- The broker NEVER passes raw privileged handles across the boundary
- The UI process NEVER renders untrusted content in consent dialogs (text is sanitized)

Boundary 2: Extension Sandbox ↔ Host

Each Wasm extension runs inside Extism/Wasmtime with WASI capability restrictions. The extension can only call host functions that the broker has approved. This boundary is enforced by the Wasm runtime — an extension literally cannot invoke a host function that wasn't linked at instantiation time.

Boundary 3: Agent Runtime ↔ Browser

The agent connects via WebSocket (JSON-RPC protocol). It has no direct access to any browser API. Every command is a message that the agent runtime routes through the broker. The agent never holds session cookies, DOM handles, or network sockets.

Boundary 4: Web Content ↔ Browser Shell

Servo renders web content in its content pipeline. The browser shell (Iced UI) is a separate rendering surface. Web content cannot draw over, obscure, or interact with the Iced UI. This prevents consent dialog spoofing.

Data Flow: Agent Action (End-to-End)

This is the canonical flow for any agent action. Every step is numbered for reference.

1. User types instruction in trusted Iced UI: "Book me a flight to London"
2. Agent Runtime receives instruction via trusted channel (not web content)
3. Agent Runtime sends instruction to LLM (Claude/GPT-4 via HTTP)
4. LLM returns action plan: [navigate to kayak.com, search flights, select cheapest, fill passenger details, click purchase]
5. For EACH action in the plan:
 - 5a. Agent Runtime constructs a CapabilityRequest:

```
{
  principal: "agent:flight-booker-v1",
  capability: "dom.write",
  scope: { origin: "https://kayak.com", tab: 3 },
  action_detail: { element: "#search-btn", action: "click" }
}
```
 - 5b. CapabilityRequest is sent to Broker via async channel
 - 5c. Broker evaluates:
 - i. Is principal registered and active? → YES/NO
 - ii. Does principal have a valid token for this capability? → YES/NO
 - iii. If no token, evaluate policy: should we grant one?
→ Regorus evaluates Rego rules against request context
 - iv. Risk classifier determines risk level:
 - "click search button" → LOW → auto-approve
 - "fill passenger name" → MEDIUM → log and allow
 - "click purchase" → HIGH → step-up consent required
 - 5d. If step-up consent required:
 - i. Broker sends ConsentRequest to UI Shell
 - ii. Iced renders consent dialog in TRUSTED UI (not in web page)
Dialog shows: "Agent wants to click 'Purchase' on kayak.com for flight LHR→SFO, \$847. Allow or Deny?"
 - iii. User clicks Allow or Deny
 - iv. Response flows back to Broker
 - 5e. Broker mints CapabilityToken (if approved) or CapabilityDenial
 - 5f. Audit System records the event:

```
{
```

```
event_id: "uuid-v7",
timestamp: "ISO-8601",
principal: "agent:flight-booker-v1",
capability: "dom.write",
scope: { origin: "https://kayak.com", tab: 3 },
action_detail: { element: "#search-btn", action: "click" },
policy_rule: "allow_navigation_low_risk",
risk_level: "low",
outcome: "granted",
token_id: "uuid-of-minted-token",
prev_hash: "sha256:...",
event_hash: "sha256:..."
}
```

5g. If granted: token is returned to Agent Runtime
Agent Runtime passes token to the engine (Servo/CEF)
to execute the action
Engine verifies token with Broker before executing

5h. If denied: denial is returned to Agent Runtime
Agent Runtime reports failure to LLM
LLM adjusts plan or reports to user

6. After all actions: audit log contains complete, verifiable record
of everything the agent did, with cryptographic integrity chain

Data Flow: Extension Action

1. Extension (Wasm module) calls host function:
`host_dom_read(selector: "div.price", origin: "https://example.com")`
2. Host function implementation (in Rust) extracts the capability
request from the function call parameters
3. Host function sends `CapabilityRequest` to Broker
4. Broker evaluates against extension's policy (loaded from manifest):
 - Extension `"price-tracker@v1"` is allowed `"dom.read"` on `"*.example.com"`
 - Rate limit: 10 requests/second (not exceeded)
 - Token not expired

5. Broker mints token → Host function receives approval

6. Host function executes DOM read via Servo API
(using Servo's content process accessors)

7. Result (sanitized DOM subtree) returned to extension

8. Audit log records: principal, capability, scope, outcome, token

If the extension tries to call `host_network_fetch()` but its manifest only declares `dom.read`:

- Host function sends `CapabilityRequest` to Broker
- Broker checks manifest: `network.fetch` not declared → DENY
- Audit log records denial with reason `"capability_not_in_manifest"`
- Host function returns error to extension
- Extension cannot retry with different parameters – the capability is fundamentally unavailable to it

Module Inventory

Module: UI Shell

- **Responsibility:** Renders browser chrome (tab bar, address bar, settings), custom DevTools panels (capability inspector, agent timeline, audit log viewer, AX tree inspector), and consent dialogs
- **Trust level:** TRUSTED
- **Framework:** Iced with dark theme
- **Inputs from:** Broker (consent requests, token status updates), Audit System (log entries for display), Servo (page titles, favicons, load status)
- **Outputs to:** Broker (consent responses, user navigation commands), Servo (navigation commands)

Module: Capability Broker

- **Responsibility:** Central authority for all capability decisions. Mints tokens, validates tokens, evaluates policies, classifies risk, manages consent flows.
- **Trust level:** TRUSTED (highest privilege component)
- **Inputs from:** Extension Sandbox (capability requests), Agent Runtime (capability requests), Network Stack (request classification), Servo Engine (page context for policy

evaluation)

- **Outputs to:** All requesters (tokens or denials), Audit System (every decision), UI Shell (consent requests)
- **Invariant:** The broker NEVER crashes the browser. All errors are handled gracefully with a default-deny outcome.

Module: Policy Engine (Regorus)

- **Responsibility:** Evaluates Rego policies against capability request context. Returns allow/deny with the rule name that matched.
- **Trust level:** TRUSTED (embedded in broker)
- **Inputs from:** Broker (policy evaluation requests with context)
- **Outputs to:** Broker (allow/deny + matching rule)
- **Invariant:** If policy evaluation fails for any reason (parse error, timeout, missing data), the result is DENY.

Module: Risk Classifier

- **Responsibility:** Categorizes capability requests as low/medium/high risk based on action type, target element, data class, and site trust tier.
- **Trust level:** TRUSTED (embedded in broker)
- **Inputs from:** Broker (capability request context)
- **Outputs to:** Broker (risk level: low/medium/high)
- **Rules:** Low = read-only actions on trusted origins. Medium = write actions on trusted origins, read on untrusted. High = anything involving credentials, payment, PII, downloads of executables, or clipboard access.

Module: Audit System

- **Responsibility:** Records every capability decision in a tamper-evident log. Provides query interface for DevTools panels. Generates and verifies cryptographic proofs.
- **Trust level:** TRUSTED
- **Storage:** Dual — append-only file (hash chain source of truth) + SQLite database (indexed queries)
- **Inputs from:** Broker (audit events), Network Stack (blocked request events)
- **Outputs to:** UI Shell (log entries for display), standalone tools (exported log files)

- **Invariant:** The audit system NEVER drops events. If disk is full, the broker pauses operations rather than losing audit data.

Module: Servo Engine (Track A)

- **Responsibility:** Renders web content. Executes JavaScript via SpiderMonkey. Performs layout and painting via WebRender + wgpu.
- **Trust level:** UNTRUSTED
- **Interface:** Via WebViewDelegate/ServoDelegate traits. Navigation commands in, page events out.
- **Invariant:** Servo never directly accesses the capability broker's internal state. All interactions go through the broker interface.

Module: CEF Harness (Track B)

- **Responsibility:** Renders web content via Chromium for compatibility comparison.
- **Trust level:** UNTRUSTED
- **Interface:** Via cef-rs bindings. Same broker interface as Servo.
- **Purpose:** Proves architecture is engine-agnostic. Used for evaluation comparisons.

Module: Extension Sandbox

- **Responsibility:** Loads and runs Wasm extensions inside Extism/Wasmtime. Provides host functions that are capability-gated.
- **Trust level:** UNTRUSTED (extensions are assumed hostile)
- **Interface:** Extism plugin API. Host functions map to capability requests.
- **Invariant:** An extension cannot discover the existence of capabilities it hasn't been granted. Host functions for ungranted capabilities are not linked at instantiation.

Module: Agent Runtime

- **Responsibility:** Accepts agent connections via WebSocket, routes JSON-RPC commands through the broker, handles consent callbacks.
- **Trust level:** UNTRUSTED (agents are assumed manipulable)
- **Interface:** JSON-RPC over WebSocket (port configurable, default 9222)
- **Invariant:** The agent runtime never holds capability tokens in memory longer than needed for a single action. Tokens are single-use or time-bounded.

Module: Network Stack

- **Responsibility:** Fetches HTTP(S) resources. Integrates adblock-rust for content blocking. Provides request classification (first-party vs third-party, tracker heuristics).
- **Trust level:** UNTRUSTED (network content is hostile)
- **Components:** hyper (HTTP/1.1+2), rustls (TLS), quinn (QUIC/HTTP3), adblock-rust (blocking)
- **Interface:** Capability-gated — network fetches by extensions/agents require network.fetch tokens.

Module: Accessibility Extractor

- **Responsibility:** Extracts accessibility tree from Servo's rendering pipeline. Provides structured AX snapshots to agents/extensions via the broker.
 - **Trust level:** Extraction is TRUSTED (runs in engine process). Output is mediated by broker.
 - **Interface:** Capability-gated — requires a11y.read token.
 - **Invariant:** AX snapshots are sanitized (no raw pointers, no internal Servo handles).
-

Cargo Workspace Structure

```
ferrite/  
├─ Cargo.toml                (workspace root)  
├─ crates/  
│   ├─ ferrite-shell/        (main binary, Iced UI, Servo/CEF integration)  
│   ├─ ferrite-broker/        (capability broker, token system, policy engine)  
│   ├─ ferrite-audit/         (audit log writer, hash chain, Merkle tree,  
SQLite)  
│   ├─ ferrite-policy/        (Regorus integration, risk classifier, Rego  
policy loading)  
│   ├─ ferrite-sandbox/       (Extism/Wasmtime host, extension loading,  
capability injection)  
│   ├─ ferrite-agent/         (JSON-RPC/WS server, command routing, consent  
callbacks)  
│   ├─ ferrite-network/       (hyper + rustls + quinn + adblock-rust  
integration)  
│   ├─ ferrite-a11y/          (AX tree extraction and serialization)  
│   ├─ ferrite-types/         (shared types: CapabilityToken, AuditEvent,  
PolicyResult, etc.)  
│   └─ ferrite-engine-trait/   (engine abstraction trait: navigate, get_dom,
```

```

execute_action)
|   └─ ferrite-servo/           (Servo WebViewDelegate implementation)
|   └─ ferrite-cef/            (CEF cef-rs implementation)
|   └─ ferrite-tools/          (CLI extension analyzer + standalone audit
inspector)
└─ policies/                   (default Rego policy files)
└─ extensions/                 (example Wasm extensions for demos)
└─ tests/                     (integration tests, adversarial test corpus)
└─ Dockerfile                 (devcontainer with all Servo deps)
└─ docs/                      (architecture docs, API docs, paper drafts)

```

Each crate has a single responsibility and minimal dependencies on other crates. `ferrite-types` is the shared vocabulary — it depends on nothing else in the workspace. All other crates depend on `ferrite-types`. The broker depends on `ferrite-policy` and `ferrite-audit`. The shell depends on everything.

Communication Patterns

Internal (within-process): Tokio async channels

- `tokio::sync::mpsc` for request/response patterns (capability requests → broker → responses)
- `tokio::sync::broadcast` for event fanout (audit events → all interested panels)
- All channels are bounded (backpressure if broker is overwhelmed → untrusted components slow down, never the broker)

External (agent protocol): WebSocket + JSON-RPC 2.0

- Agent connects to `ws://localhost:9222/agent`
- Standard JSON-RPC 2.0 with method names like `navigate`, `dom.read`, `dom.write`, `network.fetch`
- Bidirectional: browser can push consent requests to agent, agent can push action proposals
- Connection authenticated via a one-time token displayed in the UI (prevents unauthorized agent connections)

DevTools (standard): Firefox Remote Debugging Protocol

- Servo's built-in support, exposed on a configurable port
- Used for standard DOM/console/network inspection

- NOT used for Ferrite-specific features (those use Iced panels)